

Syntax & data types

COMPLETE LEARNING GUIDE

What You'll Learn

Java's type system is the foundation every later topic in this program builds on, and it is also one of the most heavily quizzed areas in placement interviews because it exposes whether a candidate truly understands how the language represents data in memory. Java is statically and strongly typed: every variable has a fixed type known at compile time, and the compiler rejects most type mismatches before the program ever runs. This guide covers the eight primitive types and their exact sizes and ranges, the crucial distinction between primitives and reference types (including autoboxing and the notorious Integer cache), widening versus narrowing conversions, the rules around the final keyword, and why String is immutable and pooled the way it is. Interviewers routinely ask "what's the difference between `==` and `.equals()`" or "why does `Integer.valueOf(127) == Integer.valueOf(127)` return true but 128 doesn't" — this section gives you the mechanics to answer both correctly and explain why.

1. The Eight Primitive Types

Java defines exactly eight primitive types, each with a fixed size that does not vary across platforms (unlike C/C++) — this platform independence is part of Java's "write once, run anywhere" guarantee. Four are integer types of increasing width (byte, short, int, long), two are floating-point (float, double), plus char (a single UTF-16 code unit) and boolean. Primitives are stored directly on the stack (for local variables) rather than as objects on the heap, which is why they are fast and have no method calls like `.toString()` available on them directly. The default numeric literal type in Java is int for whole numbers and double for decimals, which is why assigning 3.14 to a float requires an explicit f suffix (3.14f) or a cast — otherwise the compiler flags a lossy-conversion error.

- byte: 8-bit signed, -128 to 127
- short: 16-bit signed, -32,768 to 32,767
- int: 32-bit signed, about -2.1 billion to 2.1 billion (default for whole-number literals)
- long: 64-bit signed, needs an L suffix for literals beyond int range (e.g. 100000000000L)
- float: 32-bit IEEE 754, needs an f suffix (3.14f); double: 64-bit IEEE 754, the default for decimal literals
- char: 16-bit unsigned, represents a single UTF-16 code unit, e.g. 'A' or '\u0041'

- boolean: true or false only — unlike C, an int can never substitute for a boolean in Java

2. Primitives vs. Reference Types, and Autoboxing

Every primitive has a corresponding wrapper class (int !' Integer, double !' Double, boolean !' Boolean, and so on) that wraps the primitive value inside an object stored on the heap, giving it methods and letting it be used in generic collections, which cannot hold primitives directly. Autoboxing is the compiler's automatic conversion from a primitive to its wrapper (int !' Integer) when an object is expected, and unboxing is the reverse; this happens silently, for example when you add an int to a `List<Integer>`. The danger is that unboxing a null wrapper throws a `NullPointerException` with no obvious cast in the source code, and repeated autoboxing inside tight loops creates significant object-allocation overhead compared to primitive arithmetic.

JAVA

```
List<Integer> scores = new ArrayList<>();
scores.add(95);           // autoboxed: int 95 -> Integer.valueOf(95)
int first = scores.get(0); // unboxed: Integer -> int

Integer maybeNull = null;
int crash = maybeNull;    // throws NullPointerException at unboxing
```

3. The Integer Cache and == vs .equals()

Because Integer, Long, Short, Byte, and Character are objects, `==` compares references (memory addresses), not values — this is the single most common source of a subtle bug for developers coming from languages where `==` compares value. The JVM caches boxed Integer values from -128 to 127 (per the `Integer.valueOf` spec) as a performance optimization, so two autoboxed Integers in that range that hold the same value will happen to point at the same cached object, making `==` appear to "work" by accident. Outside that range, `==` reliably returns false for two separately created Integer objects with the same value, which is exactly why interviewers love this question — it exposes whether you understand reference identity versus value equality, not just memorized behavior.

- `Integer.valueOf(100) == Integer.valueOf(100) !' true` (both within the cached -128..127 range)
- `Integer.valueOf(200) == Integer.valueOf(200) !' false` (outside the cache, two distinct objects)
- `Integer.valueOf(100).equals(Integer.valueOf(100)) !' always true` — `.equals()` compares value
- Rule of thumb: always use `.equals()` (or compare unboxed primitives with `==`) for wrapper types, never `==`

4. Widening, Narrowing, and Casting

Widening conversions (byte !' short !' int !' long !' float !' double) happen implicitly because no information can be lost, so Java performs them automatically wherever a wider type is expected. Narrowing conversions (the reverse direction, e.g. double !' int) can lose data or precision, so the compiler forces an explicit cast to make the potential loss visible in the source code. Casting a double to an int truncates the fractional part rather than rounding — (int) 9.99 evaluates to 9, not 10 — and casting an out-of-range long to an int silently wraps around using two's-complement overflow rather than throwing an exception, which is a frequent source of quietly wrong results in interview code.

JAVA

```
int i = 100;
long l = i;           // widening, implicit
double d = l;         // widening, implicit

double price = 9.99;
int truncated = (int) price; // 9, not 10 -- truncation, not rounding

long big = 3_000_000_000L;
int overflowed = (int) big; // wraps around, NOT 3,000,000,000
```

5. String Immutability and the String Pool

Strings in Java are immutable — once created, a String object's contents can never change; every "mutating" method (substring, concat, replace, toUpperCase) returns a brand-new String object rather than modifying the original. This immutability is what makes String safe to share across threads without synchronization and safe to use as a HashMap key (its hashCode can be cached once). String literals are additionally interned in a special memory region called the string pool: two literals with identical text share the same object, so "abc" == "abc" is true, but new String("abc") explicitly forces heap allocation of a new object outside the pool, so new String("abc") == "abc" is false even though .equals() returns true for both.

- String literals are pooled and reused: "abc" == "abc" !' true
- new String("abc") forces a new heap object: new String("abc") == "abc" !' false
- .intern() manually places (or retrieves) a String in the pool
- Because Strings are immutable, repeated concatenation in a loop (s += x) creates a new object every iteration — use StringBuilder instead

6. Type Inference with var

Since Java 10, the var keyword lets the compiler infer a local variable's type from its initializer, reducing boilerplate for verbose generic types (var map = new HashMap<String, List<Integer>>();) while remaining fully statically typed — var is not dynamic typing, and the inferred type is fixed permanently at compile time. var can only be used for local variables with an initializer present (never for fields, method parameters, or return types), and it cannot be used with a null literal alone since there would be nothing to infer from. Overusing var when the right-hand side doesn't make the type obvious (e.g. var result = process();) hurts

readability, so most style guides recommend it mainly when the type is already explicit in the initializer, such as constructor calls.

7. final Variables and Constants

The `final` keyword applied to a variable means it can be assigned exactly once — for a primitive, that fixes its value permanently; for a reference type, it fixes which object the variable points to, but the object's own internal state can still change if that object is mutable (final `List<Integer> list` can still have elements added to it, even though `list` itself can never be reassigned). By convention, class-level constants combine `static` and `final` (public static final `int MAX_USERS = 100;`) so the value is shared once per class rather than duplicated per instance, and uppercase-with-underscores naming signals immutability to readers at a glance.

Common Pitfalls

- Using `==` to compare boxed `Integer/Long/Character` objects and getting inconsistent results depending on whether the value falls inside the `-128..127` cache range.
- Assuming `(int) someDouble` rounds — it truncates toward zero, so `(int) 9.99` is 9, not 10.
- Forgetting the `L` suffix on a long literal that exceeds `int` range, causing a compile-time "integer number too large" error or silent overflow.
- Concatenating Strings with `+=` inside a loop, unaware that each iteration allocates a brand-new `String` object because Strings are immutable.
- Unboxing a null wrapper object (e.g. assigning a null `Integer` field to an `int`) and getting a `NullPointerException` with no visible cast at the crash site.
- Believing `new String("x") == "x"` should be true because the text is identical — it compares references, and `new String(...)` is deliberately taken out of the pool.
- Declaring a field or method parameter with `var`, not realizing `var` is legal only for local variables with an initializer.

Key Takeaways

- Java has exactly eight primitives with fixed, platform-independent sizes — memorize `byte/short/int/long/float/double/char/boolean` and their ranges.
- Always compare wrapper objects with `.equals()`, never `==` — the `Integer` cache (`-128` to `127`) makes `==` unreliable and inconsistent.
- Widening conversions are implicit and safe; narrowing conversions require an explicit cast and can truncate or overflow silently.
- `String` is immutable — every "mutating" method returns a new object, and string literals are pooled while `new String(...)` is not.
- `var` is compile-time type inference, not dynamic typing — the type is fixed forever at the point of declaration.

- final on a reference variable freezes which object it points to, not the mutability of that object's internal state.

Learn More

Oracle Java Tutorials — Primitive Data Types

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>

Oracle Java Tutorials — Autoboxing and Unboxing

<https://docs.oracle.com/javase/tutorial/java/data/autoboxing.html>

GeeksforGeeks — Java String Pool

<https://www.geeksforgeeks.org/java/string-pool-java/>